Programming and Data Structures
Programming Assignment 3
Recursion and Algorithm Analysis

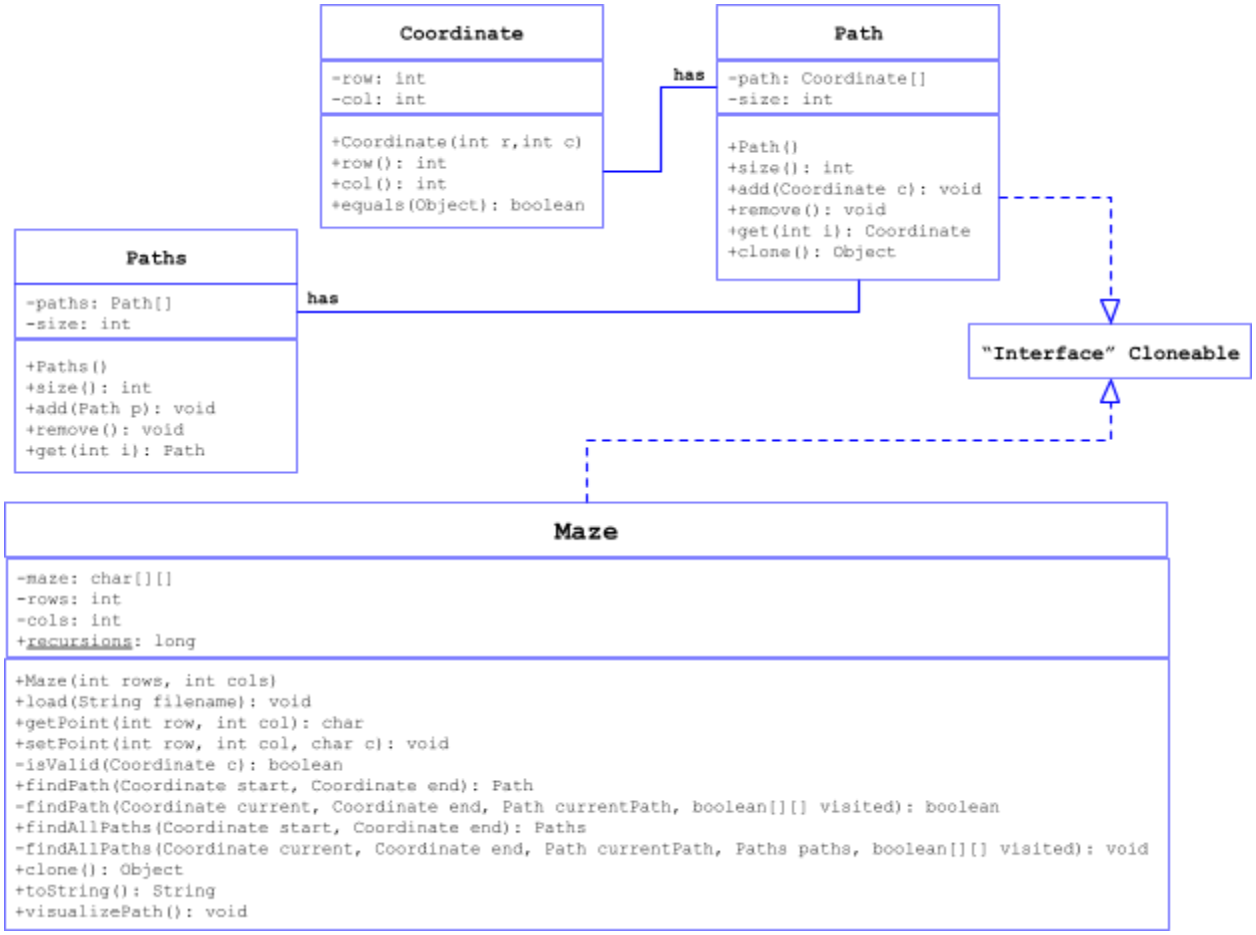
Assignment Learning Objectives

At the end of this assignment, students should be able to:

1. Write recursive methods to solve a given problem.
2. Analyze the time complexity of recursive methods.

Description

This programming assignment is a comprehensive exercise in **recursion** and **memory management**. You are building a system that solves a maze and manages the data structures required to track every possible solution. Implement the classes shown in the UML diagram below:



The assignment uses a custom collection hierarchy rather than standard Java **`ArrayLists`**. This means you are responsible for manual array management.

1. Class **`Coordinate`**
    a. A basic point (row,col).
    b. **`equals()`**: The method returns true if two coordinates have the same row and column.

2. Class **`Path`**
    a. The constructor creates the array **`path`** with capacity 200 and initializes **`size`** to 0.
    b. **`add()`**: adds the given coordinate at index **`size`** and increments **`size.`**
    c. **`remove()`**: removes the coordinate at index **`size-1`** by decrementing **`size.`**
    d. **`get()`**: returns the coordinate at the given index or throws an `ArrayIndexOutOfBoundsException` if the given index is not valid.
    e. **`clone()`**: returns a deep copy on this path.

3. Class **`Paths`**
    a. The constructor creates the array **`paths`** with an initial capacity of 10,000 and initializes **`size`** to 0.
    b. **`addPath()`**: adds the given path at index **`size`** and increments **`size.`**
    c. **`remove()`**: removes the path at index **`size-1`** by decrementing **`size`**.
    d. **`get()`**: returns the path at the given index or throws an `ArrayIndexOutOfBoundsException` if the given index is not valid.

4. Class **`Maze`**
    a. The constructor creates the two dimensional array **`maze`** with the given dimensions.
    b. **`load()`**: reads (rows x cols) characters from the given file and stores them in the array maze (row-major format, see the sample mazes in maze_x.txt files. 'S' for start, 'E' for exit, '.' for free way, and '#" for wall or closed way).
    c. **`getPoint()`**: returns the character at the given row and column, or throws an ArrayIndexOutOfBoundsException if the given row or column are out of bounds.
    d. **`setPoint()`**: sets the location given by the row and column to the given character. It throws an ArrayIndexOutOfBoundsException if the given row or column are not valid.
    e. **`isValid()`**: returns true if the given coordinate is within the bounds of the maze, false otherwise.

f. **`findPath()`**: returns the path from the given start to the given end in the maze, null if no path was found. This method is recursive and uses a helper method described below. The method passes the coordinates start and end, a new Path object, and a two-dimensional array of booleans to the helper method to keep track of the visited points in the maze.

g. **`Recursive findPath()`**: finds a single path from the current position to end. The found path is returned in the currentPath argument. The method first checks the current position for the following base cases:
   i. The current position is not valid (reached the bounds of the maze), return false
   ii. The current position is a wall ('#'), return false
   iii. The current position has already been visited, return false
   iv. The current position is equal to **`end`**, add it to the current path and return true

   If none of the base cases was checked, set the visited boolean of the current position to true and call recursively the methods with next locations:
   v. Recursive call with a new coordinate (right). If a path is found return true
   vi. If no path is found on the right, make a recursive call with a new coordinate (left). If a path is found, return true.
   vii. If no path is found on the left, make a recursive call with a new coordinate (up). If a path is found, return true.
   viii. If no path is found on the up, make a recursive call with a new coordinate (down). If a path is found, return true.
   ix. remove the current position from the current path and reset its visited boolean to false if all directions return false, then return false

h. **`findAllPaths`**: returns the list of all possible paths from the current position to end. The method is recursive and uses a helper method described below. The method passes the following arguments to the helper method: the coordinates start and end, a new Paths object to hold the list of paths, a two-dimensional array to keep track of the visited coordinates, and a new Path object for the current path.

i. **`Recursive findAllPaths`**: finds all paths from the current position to end. The list of paths found is returned in the **`Paths`** object. The method first checks the current position for the following base cases.
   i. The current position is not valid (reached the bounds of the maze), return.
   ii. The current position is a wall ('#'), return.

iii. The current position has already been visited, return.
iv. The current position is equal to end, add it to the current path, set its visited boolean to true, and add the current path to the list of paths.

Otherwise, call recursively the method with new positions (right, left, up, and down.

Remove the current position from the current path and reset its visited status to false before returning from the method so that other potential paths can use that same coordinate.

j. **`toString()`**: returns a formatted string that contains the two-dimensional array of characters organized in rows and columns.
k. **`clone()`**: returns a deep copy of this maze.
l. **`visualizePath()`**: displays the given path on the maze. This method clones this maze, modifies the clone by filling the cells corresponding to the given path with the marker '*', then displays the cloned maze. Ensure the clone method creates a deep copy of the maze, otherwise, the **`*`** markers will ruin your original maze data.

5. **Time Complexity Analysis**
   a. Determine the time complexity of all the methods in the class Maze. Write the Big-O notation as a comment before the header of each method.
   b. Count the number of recursive calls in the methods **`findPath`** and **`findAllPaths`** using the static variable **`recursions`**.
   c. Discuss the output results for test case 5.

6. Testing and Submission

The Starter Code contains:

a. **`Test.java`** to test your classes.
b. **`runTests.sh`** to compare your program output to `output.reference`.

   Run this in the terminal using: `bash runTests.sh`
c. Sample maze text files (maze_1.txt to maze_5.txt) used for testing.
d. `output.reference`: the expected output of the program.

Submission: Submit the files: **`Coordinate.java`**, **`Path.java`**, **`Paths.java`**, and **`Maze.java`** on Gradescope.